L05 – Multilayer Perceptron (MLP)

Foundations Lecture Series

Course Instructor

Microgrid 3-phase Security AI Project

Foundations Lecture 5

Keywords: linear → nonlinear, layers, activations, backprop, train/val/test, PyTorch

Lecture roadmap

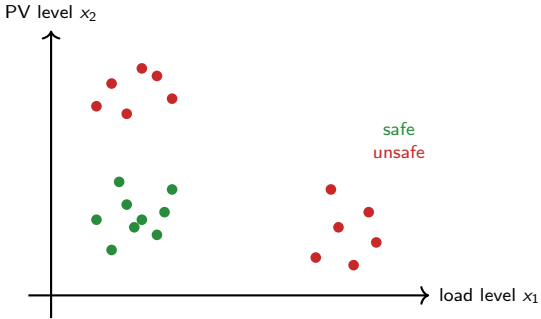
1. Why MLPs: a problem linear models cannot solve
2. Anatomy of an MLP: layers, weights, activations
3. Output heads for classification, regression, multi-task
4. Training: loss, backprop, gradient descent
5. Discipline: train/val/test splits, regularization, class imbalance
6. A minimal PyTorch example, end to end
7. Common pitfalls and how the project uses MLPs

Prerequisite

Basic linear algebra (matrix-vector products), basic calculus (partial derivatives, chain rule), and Python literacy. No prior ML experience required.

A toy classification problem, in our project's spirit

For every microgrid operating state we want to predict: *will any contingency cause a violation?* A 2D toy version captures the geometry:

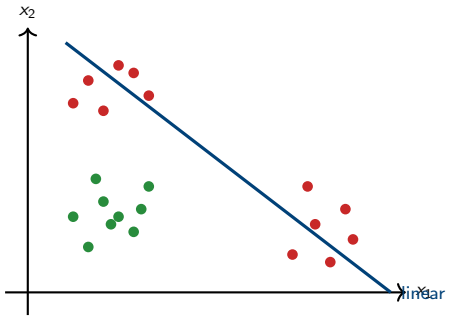


The geometry

Two "unsafe" clusters: high load on the right (overload) and high PV on the top (reverse-flow voltage rise). A single straight line cannot separate them from the safe cluster in the middle.

Linear classifier: what it can and cannot do

A linear classifier predicts $\hat{y} = \sigma(w_1 x_1 + w_2 x_2 + b)$ – the decision boundary is a single straight line



$w_1 x_1 + w_2 x_2 + b = 0.$

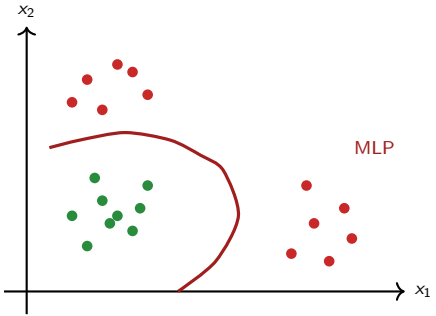
What's wrong

The best linear separator must *trade off*: it can correctly classify the two unsafe clusters but only by also flagging some safe points as unsafe, or vice versa. No single line works for this geometry. Accuracy plateaus around 75%.

MLP: stack nonlinear layers, get a curved boundary

An MLP composes *affine maps* with *nonlinear activations*:

$$h_{l+1} = \sigma(W_l h_l + b_l), \quad l = 0, 1, \ldots, L-1$$

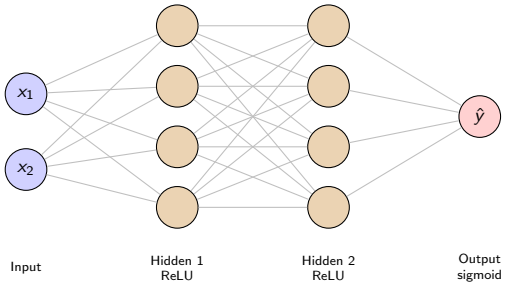


Why the curved boundary is possible

Each layer applies an affine map (so far still linear), *then* a nonlinearity $\sigma(\cdot)$. Stacking these compositions builds curved decision regions. The capacity grows with depth and width.

The architecture diagram

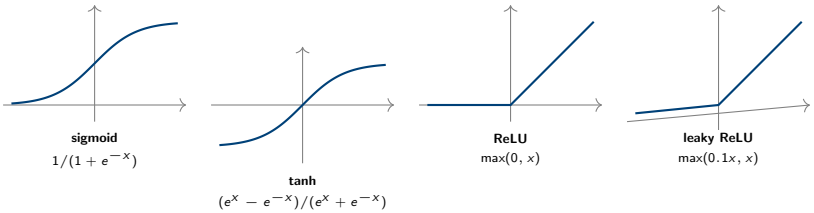
A 2-hidden-layer MLP for our 2D toy problem:



What each piece is

- Each circle is a *neuron*, i.e., one scalar computation $\sigma(\sum_j w_j x_j + b)$.
- Each gray line is one learnable weight w_{ij}. Counting: $2 \cdot 4 + 4 \cdot 4 + 4 \cdot 1 = 28$ weights, plus 9 biases.
- Width (e.g., 4 here) and depth (e.g., 2 hidden) are hyperparameters.

Activation functions



Practical defaults

- Hidden layers: **ReLU** (default, fast, works almost everywhere)
- Binary output: **sigmoid** (squashes to $[0, 1]$)
- Multi-class output: **softmax**
- Regression output: **identity** (no activation)

Output heads for different tasks

Task	Output activation	Loss
Binary classification (safe vs unsafe)	sigmoid	binary cross-entropy (BCE)
Multi-class classification (5 violation types)	softmax	cross-entropy
Regression (severity score)	identity / linear	mean squared error (MSE)
Ranking (top-k risk order)	identity	pairwise hinge / ranking loss
Multi-task (project's PI-MLP)	multiple heads at once	weighted sum of the above

What "multi-task" looks like architecturally

The shared layers extract a common feature representation. Then multiple heads branch off, each with its own loss. The total loss is $\mathcal{L} = \lambda_1 \mathcal{L}_{\text{cls}} + \lambda_2 \mathcal{L}_{\text{sev}} + \lambda_3 \mathcal{L}_{\text{rank}} + \ldots$. The project's Week 6 PI-MLP does exactly this.

Choosing width and depth

Rules of thumb

- Start small: 1–2 hidden layers, 32–128 units.
- Train. Look at train and validation loss.
- Underfitting (both losses high): widen or deepen.
- Overfitting (train low, val high): shrink or regularize.

The project's regime

- Only 77 labeled samples.
- Too much capacity $\rightarrow$ memorize the training set, generalize poorly.
- Week 5/6 use small MLPs ($\sim$ 32 hidden units, 2 layers) with weight decay and early stopping.

Don't reach for a 10-layer transformer

On a small tabular dataset like ours, simple is competitive. Modern research often finds that gradient boosting beats deep nets on small tabular data – and that a well-tuned 2-hidden-layer MLP beats most things below 1000 samples.

The training loop in one slide

1. Pick a minibatch of data (X, y) from the training set.
2. **Forward pass**: compute predictions $\hat{y} = f_\theta(X)$.
3. **Loss**: compute $\mathcal{L}(\hat{y}, y)$ – a scalar measuring how wrong we are.
4. **Backward pass**: compute $\nabla_\theta \mathcal{L}$ via the chain rule.
5. **Update**: $\theta \leftarrow \theta - \eta \, \nabla_\theta \mathcal{L}$ (gradient descent step).
6. Go to 1. Repeat for many epochs until validation loss stops improving.

Three knobs in this loop

Batch size (how many samples per step), **learning rate** η (step size), and **number of epochs** (passes through the data). Tuning these is most of the practical art.

Common loss functions

Binary classification. Output $\hat{p} \in [0,1]$ (sigmoid), target $y \in \{0,1\}$:

$$\mathcal{L}_{\text{BCE}} \;=\; -\big[y \log \hat{p} + (1-y)\log(1-\hat{p})\big]$$

Penalizes confident-wrong predictions heavily.

Regression. Output $\hat{y}$, target y:

$$\mathcal{L}_{\text{MSE}} \;=\; (\hat{y} - y)^2$$

Multi-class classification. Output probabilities $\hat{p}_k$ over K classes:

$$\mathcal{L}_{\text{CE}} \;=\; -\sum_{k} y_k \log \hat{p}_k$$

Class-weighted BCE for imbalance

$$\mathcal{L}_{\text{wBCE}} = -\big[\, w_1 \cdot y \log \hat{p} + w_0 \cdot (1-y)\log(1-\hat{p})\big]$$

Set $w_1 > w_0$ if the positive (unsafe) class is rarer than the negative. The project uses this for screening recall.

Backpropagation in one slide

The training step needs $\partial \mathcal{L} / \partial W_l$ for every layer. By the chain rule:

$$\frac{\partial \mathcal{L}}{\partial W_l} = \frac{\partial \mathcal{L}}{\partial h_L} \cdot \frac{\partial h_L}{\partial h_{L-1}} \cdots \frac{\partial h_{l+1}}{\partial W_l}$$

- Each factor is a small Jacobian computable in closed form for ReLU / sigmoid / tanh / linear layers.
- Multiplying them out from the output back toward the input is the "backward pass".
- **PyTorch's autograd does this for you.** You never derive gradients by hand.

The one rule

You write the forward pass. Calling `loss.backward()` fills in `param.grad` for every parameter. You then call `optimizer.step()`. That's it – no manual calculus.

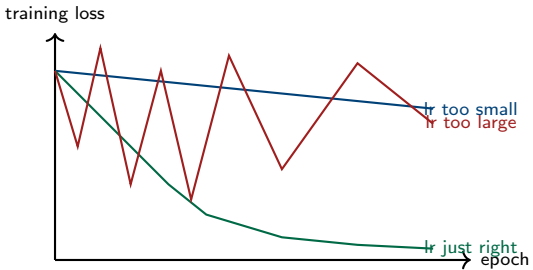
Optimizers: SGD, momentum, Adam

Optimizer	Update rule (per parameter θ)	When to use
SGD	$\theta \leftarrow \theta - \eta \cdot \nabla \mathcal{L}$	simple, well-tuned baseli
SGD + momentum	$v \leftarrow \mu v + \nabla \mathcal{L}$; $\theta \leftarrow \theta - \eta v$	smoother trajectories
Adam	per-parameter adaptive η using 1st and 2nd moment estimates	default choice today

Recommended default for this course

`torch.optim.Adam(model.parameters(), lr=1e-3)`. Works well on $\sim 90\%$ of small-to-medium problems. For very small datasets like ours, you may need to drop to lr $= 10^{-4}$ to avoid oscillation.

Learning rate: the most important hyperparameter



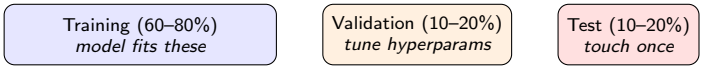
Symptoms

- Too small: loss barely moves.
- Just right: loss drops smoothly to a plateau.
- Too large: loss oscillates, may even diverge.

Practical approach

- Start at $\text{lr} = 10^{-3}$ for Adam.
- If loss explodes, reduce by $10\times$.
- If loss plateaus too high, increase by $3\times$.
- Optionally use a scheduler.

Train / validation / test – and why all three



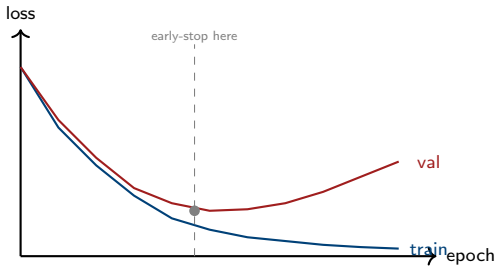
The cardinal rule

The **test set is touched once**, at the very end of the project, to estimate generalization. *Every* hyperparameter (learning rate, hidden width, regularizer strength, decision threshold) is chosen using validation, not test.

Why validation is separate from test

If you tune on the test set, the test number becomes meaningless – you have implicitly learned from it. The project uses *group* cross-validation (by scenario and by contingency), not random splits, to avoid leakage between similar samples.

Overfitting and the train-val gap



The inflection point matters

Training loss keeps dropping forever; validation loss eventually starts *rising*. The point where validation bottoms out is the best generalizing model. Early stopping = save the model checkpoint at that minimum.

The regularization toolbox

Technique	Mechanism	When to reach for it
L2 weight decay	add $\lambda\|\theta\|^2$ to the loss	always – almost free
Dropout	randomly zero out activations during training	medium / large nets
Early stopping	stop at the val-loss minimum	always – almost free
Data augmentation	synthetic variations of training data	images, time series
Smaller model	reduce width or depth	quick and effective

For the project

L2 weight decay + early stopping is enough. The dataset is small enough that dropout often hurts (further reducing effective capacity). Week 5/6 notebooks use $\lambda \approx 10^{-4}$ and a patience of 30 epochs.

Class imbalance and the right metric

The project's labeled dataset: **47 unsafe, 30 safe** (61% positive).

Accuracy is the wrong metric

A model that predicts "always unsafe" achieves 61% accuracy on training set. A model that's right on 9 out of 10 unsafe but wrong on every safe one achieves $\sim 75\%$ accuracy. Neither is useful.

Right metrics for safety screening

- **Recall** on unsafe class
- **FNR**: 1 – recall (lower is better)
- **AUPRC**: area under precision-recall curve
- **Top-k hit rate**: of the k most-suspicious predictions, how many are truly unsafe?

Techniques

- Class-weighted BCE
- Decision-threshold tuning (lower threshold favours recall)
- Focal loss (advanced; usually unnecessary at our scale)

Step 1: data and model

```
import torch
import torch.nn as nn
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split

# Toy data: nonlinearly separable 2D binary classification
X, y = make_moons(n_samples=400, noise=0.20, random_state=0)
X_tr, X_va, y_tr, y_va = train_test_split(X, y, test_size=0.25, random_state=0)
X_tr = torch.tensor(X_tr, dtype=torch.float32)
y_tr = torch.tensor(y_tr, dtype=torch.float32).unsqueeze(1)
X_va = torch.tensor(X_va, dtype=torch.float32)
y_va = torch.tensor(y_va, dtype=torch.float32).unsqueeze(1)

# 2 -> 32 -> 32 -> 1 MLP with sigmoid output
model = nn.Sequential(
    nn.Linear(2, 32), nn.ReLU(),
    nn.Linear(32, 32), nn.ReLU(),
    nn.Linear(32, 1), nn.Sigmoid(),
)
```

Step 2: training loop

```
opt = torch.optim.Adam(model.parameters(), lr=1e-3, weight_decay=1e-4)
bce = nn.BCELoss()

best_val = float("inf")
for epoch in range(200):
    # --- train ---
    model.train()
    opt.zero_grad()
    loss = bce(model(X_tr), y_tr)
    loss.backward()
    opt.step()

    # --- validate ---
    model.eval()
    with torch.no_grad():
        val_loss = bce(model(X_va), y_va).item()

    if val_loss < best_val:
        best_val = val_loss
        torch.save(model.state_dict(), "best.pt") # early-stopping checkpoint
```

What this does

One forward pass on the whole training set per epoch (full-batch, fine for $n = 300$), Adam update, validation check, save best model.

Step 3: evaluate and pick the decision threshold

```python
import numpy as np
from sklearn.metrics import confusion_matrix, precision_recall_curve, auc

model.load_state_dict(torch.load("best.pt"))
model.eval()
with torch.no_grad():
    p_va = model(X_va).squeeze().numpy()

# Threshold to maximize recall while keeping precision >= 0.85.
# Note: precision_recall_curve returns len(thr) = len(prec) - 1, so we align with prec[:-1].
prec, rec, thr = precision_recall_curve(y_va.squeeze().numpy(), p_va)
valid = prec[:-1] >= 0.85 # aligned with thr
threshold = thr[valid][np.argmax(rec[:-1][valid])] if valid.any() else 0.5

y_pred = (p_va >= threshold).astype(int)
tn, fp, fn, tp = confusion_matrix(y_va.squeeze().numpy(), y_pred).ravel()
print(f"recall={tp/(tp+fn):.3f} precision={tp/(tp+fp):.3f} AUPRC={auc(rec, prec):.3f}")
```

Decision threshold is a hyperparameter

Choose it on the validation set, never on test. For screening: prefer thresholds that keep recall high (low FNR), even if precision drops – false alarms are cheaper than missed violations.

Common pitfalls in MLP training

1. △ **Inputs not normalized.** Features with very different scales (e.g., voltage in [0.9, 1.1] vs power in [0, 100]) make optimization slow. Always standardize.
2. △ **Wrong output activation.** Sigmoid before BCELoss, but *not* before `BCEWithLogitsLoss`. Mixing them up either breaks training or hides bugs.
3. △ **Data leakage** between train and test. Feature engineering on the full dataset, then splitting, leaks information. Fit scalers / encoders on train only.
4. △ **Tuning on the test set.** You will overfit to it and report optimistic numbers.
5. △ **Reporting accuracy on imbalanced data.** The constant predictor often gets a high accuracy. Report recall / FNR / AUPRC.
6. △ **Forgetting to seed.** Set `torch.manual_seed(0)` and the numpy and sklearn seeds to make experiments reproducible.

How the project uses MLPs

Week 5: ML baselines

- Plain MLP as one of several baseline classifiers.
- Inputs: scenario features + contingency descriptor.
- Output: binary "will this contingency cause a violation?"
- Compared against logistic regression, random forest, gradient boosting, engineering rule.

Week 6: Physics-informed MLP

- Same MLP backbone, multiple output heads (classification + severity + component margins).
- Loss = data BCE + physics penalty (voltage / current / VUF limit violations).
- Goal: keep recall high while gaining interpretability.
- Lecture L07 will develop the PI-MLP idea in detail.

What you learn here that you reuse

Every concept in L05 – architecture, training loop, regularization, class imbalance, threshold tuning – is reused in Week 5 and Week 6 of the project. Master them on this toy 2D problem and the project notebooks become *very* readable.

Homework

Required

1. Reproduce the `make_moons` example above. Verify your final validation recall and AUPRC.
2. Train the same MLP with random initial seeds in $\{0, 1, 2, 3, 4\}$. Report mean and stdev of recall.
3. Replace the toy data with a *class-imbalanced* version (use `sklearn` to drop 80% of one class). Re-train with *plain* BCE and then with *class-weighted* BCE ($w_1 = 5$). Compare recall.
4. Plot the trained decision boundary by evaluating the model on a 100×100 grid over $[-2, 3] \times [-1, 2]$.

Optional

1. Replace ReLU with tanh. Retrain. Does anything change?
2. Add a third hidden layer. Does it help on this small dataset?
3. Implement early stopping with patience $= 30$ epochs and compare to fixed-200-epoch training.

Exit checklist

1. ✓ I can explain why a single linear classifier cannot solve XOR / moons.
2. ✓ I can write the equation $h_{l+1} = \sigma(W_l h_l + b_l)$ and explain each symbol.
3. ✓ I can pick the right output activation + loss for binary, multi-class, regression.
4. ✓ I can write a PyTorch training loop in ~ 15 lines from memory.
5. ✓ I can explain what backprop does without deriving any gradients by hand.
6. ✓ I can read a train-val loss curve and identify the overfitting onset.
7. ✓ I can name three reasons accuracy is the wrong metric for safety screening.
8. ✓ I am ready for L06 (GNN) and Week 5/6 of the project.

References

- **Goodfellow, Bengio, Courville,** *Deep Learning* (MIT Press, 2016). Chapters 6 (MLPs) and 7 (regularization). Free online.
- **Bishop,** *Pattern Recognition and Machine Learning* (Springer, 2006). Chapter 5 on neural networks, with a strong probabilistic flavor.
- **Murphy,** *Probabilistic Machine Learning: An Introduction* (MIT Press, 2022). Modern treatment, free online.
- **PyTorch official tutorials:** `https://pytorch.org/tutorials/` – "Learn the Basics" and "60 Minute Blitz".
- **Karpathy,** *A Recipe for Training Neural Networks* (blog, 2019). The single best practical guide.
- **scikit-learn** documentation: `https://scikit-learn.org/stable/user_guide.html` for data, splits, metrics.